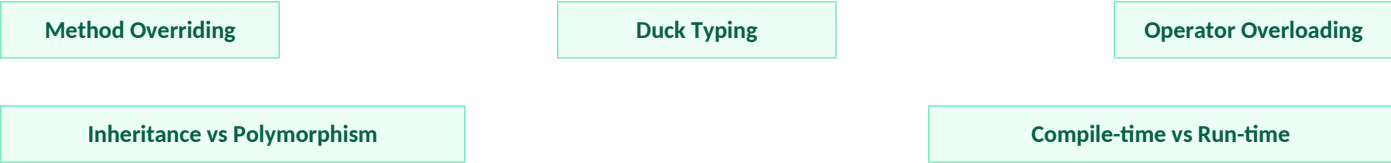


Polymorphism:

One Interface, Many Forms

A visual, example-driven reference covering method overriding, duck typing, the relationship between inheritance and polymorphism, operator overloading, and the difference between compile-time and run-time behaviour — illustrated with animals, payment systems, and game characters.



1. What Is Polymorphism?

Polymorphism is one of the four main pillars of Object-Oriented Programming, alongside encapsulation, inheritance, and abstraction. The word breaks down neatly into its Greek roots.

Poly = Many **Morph** = Forms

Put together, polymorphism means “one interface, many forms.” The same method or function call can behave differently depending on the object that receives it.

REAL-LIFE ANALOGY — THE REMOTE CONTROL

You press the same Power button every time, but the result depends on what the remote is pointed at:

- Pointed at the TV → the TV turns on
- Pointed at the AC → the AC turns on
- Pointed at the Fan → the Fan turns on

The button (interface) never changes. The behaviour changes depending on the device. That is polymorphism.

2. Life Without Polymorphism

Consider three animal classes, each with its own uniquely named method:

`animals_no_poly.py`

```
class Dog:
    def bark(self):
        print("Dog says Woof!")

class Cat:
    def meow(self):
        print("Cat says Meow!")

class Cow:
    def moo(self):
        print("Cow says Moo!")

dog = Dog()
cat = Cat()
cow = Cow()

dog.bark()
cat.meow()
cow.moo()
```

OUTPUT

```
Dog says Woof!
Cat says Meow!
Cow says Moo!
```

► The problem starts to show

Now imagine there are 100 different animals — `lion.roar()`, `duck.quack()`, `horse.neigh()` — every one has a differently named method. A loop can't call them uniformly, because there is no shared method name:

`no_common_interface.py`

```
animals = [dog, cat, cow]

for animal in animals:
    if isinstance(animal, Dog):
        animal.bark()
    elif isinstance(animal, Cat):
        animal.meow()
    elif isinstance(animal, Cow):
        animal.moo()
```

WHY THIS HURTS

Every time a new animal type is added, this if/elif chain has to grow. The caller has to know about every possible subclass just to use it — that's fragile, repetitive, and hard to maintain.

3. The Same Problem, With Polymorphism

Give every class the same method name instead of a different one each:

animals_with_poly.py

```
class Dog:
    def sound(self):
        print("Dog says Woof!")

class Cat:
    def sound(self):
        print("Cat says Meow!")

class Cow:
    def sound(self):
        print("Cow says Moo!")

dog = Dog()
cat = Cat()
cow = Cow()

dog.sound()
cat.sound()
cow.sound()
```

OUTPUT

```
Dog says Woof!
Cat says Meow!
Cow says Moo!
```

Same output — but now every object answers to the identical method call, `sound()`. That single change is what makes polymorphism possible.

► Even better, combined with inheritance

animals_inheritance_poly.py

```
class Animal:
    def sound(self):
        print("Some sound")

class Dog(Animal):
    def sound(self):
        print("Woof!")

class Cat(Animal):
    def sound(self):
        print("Meow!")

class Cow(Animal):
    def sound(self):
        print("Moo!")
```

```
animals = [Dog(), Cat(), Cow()]
```

```
for animal in animals:  
    animal.sound()
```

OUTPUT

Woof!

Meow!

Moo!

WHAT JUST HAPPENED

The loop never asks “is this a Dog? a Cat? a Cow?” — it simply calls `animal.sound()` and Python automatically runs the correct version for each object. This is run-time polymorphism.

4. A Larger Example — Payment Processing

► Without polymorphism

payment_no_poly.py

```
class CreditCard:
    def pay_by_card(self, amount):
        print(f"Paid ${amount} using Credit Card")

class PayPal:
    def pay_by_paypal(self, amount):
        print(f"Paid ${amount} using PayPal")

class BankTransfer:
    def pay_by_bank(self, amount):
        print(f"Paid ${amount} using Bank Transfer")

method = card

if isinstance(method, CreditCard):
    method.pay_by_card(500)
elif isinstance(method, PayPal):
    method.pay_by_paypal(500)
elif isinstance(method, BankTransfer):
    method.pay_by_bank(500)
```

PROBLEMS WITH THIS APPROACH

- Every class uses a different method name
- A long, growing if/elif chain
- Hard to maintain as more payment types arrive
- Adding a new payment method means editing this logic

► With polymorphism

payment_with_poly.py

```
class Payment:
    def pay(self, amount):
        pass

class CreditCard(Payment):
    def pay(self, amount):
        print(f"Paid ${amount} using Credit Card")

class PayPal(Payment):
    def pay(self, amount):
        print(f"Paid ${amount} using PayPal")

class BankTransfer(Payment):
```

```
def pay(self, amount):
    print(f"Paid ${amount} using Bank Transfer")

payments = [CreditCard(), PayPal(), BankTransfer()]

for payment in payments:
    payment.pay(500)
```

OUTPUT

```
Paid $500 using Credit Card
Paid $500 using PayPal
Paid $500 using Bank Transfer
```

No if, no elif, no type-checking — each object simply knows how to perform its own pay().

► Why this scales so well

Adding Bitcoin support means writing one new class and appending it to the list — the processing loop itself never changes:

payment_add_bitcoin.py

```
class Bitcoin(Payment):
    def pay(self, amount):
        print(f"Paid ${amount} using Bitcoin")

payments.append(Bitcoin())

for payment in payments:
    payment.pay(500)
```

This is the Open/Closed Principle in action: the code is open for extension (new payment types can be added freely) but closed for modification (the payment-processing loop never has to change).

Side-by-Side Comparison

Without Polymorphism	With Polymorphism
Different method names (bark(), meow(), moo())	Same method name (sound())
Requires if/elif or type checking	No type checking needed
Hard to extend	Easy to add new classes
Caller must know every class	Caller only knows the common interface
More repetitive code	Cleaner, more reusable code

5. Polymorphism vs. Inheritance

Polymorphism very often works together with inheritance, which is exactly why beginners tend to mix the two up — but they solve different problems:

Inheritance lets different classes share a common structure and behaviour.

Encapsulation protects data and controls how it is accessed or modified.

Polymorphism lets you call the same method on different objects, with each object performing its own version of that method.

The Key Distinction

Inheritance	Polymorphism
Creates a relationship between classes	Changes behaviour of the same method
Used for code reuse	Used for flexibility
An “is-a” relationship	“One interface, many behaviours”
Parent → Child	Same method → different implementations
Answers “What is this object?”	Answers “How should this object behave?”

► Think of it like a family tree



Dog is-an Animal. Cat is-an Animal. Cow is-an Animal — that relationship is inheritance. Now suppose every animal can make a sound: Dog.sound() → Woof!, Cat.sound() → Meow!, Cow.sound() → Moo! The method name stays the same, but each animal behaves differently — that is polymorphism.

► Example A — inheritance only, no polymorphism

```
inheritance_only.py

class Animal:
    def eat(self):
        print("Animal is eating")

class Dog(Animal):
    pass

dog = Dog()
dog.eat()
```

OUTPUT

Animal is eating

Dog simply inherited eat() unchanged — nothing was overridden, so there is no polymorphic behaviour here.

► Example B — inheritance plus polymorphism

inheritance_plus_poly.py

```
class Animal:
    def sound(self):
        print("Some sound")

class Dog(Animal):
    def sound(self):
        print("Woof!")

class Cat(Animal):
    def sound(self):
        print("Meow!")

dog = Dog()
cat = Cat()

dog.sound()
cat.sound()
```

OUTPUT

```
Woof!
Meow!
```

Inheritance created the parent-child relationship; each child then overrides `sound()` with its own version. Calling the same method name and getting different results — that's polymorphism riding on top of inheritance.

EVERYDAY EXAMPLE — THE SMARTPHONE

Inheritance: every smartphone has a power button, a screen, and a battery — a Samsung phone inherits these general smartphone features.

Polymorphism: press the camera button — on a Samsung it opens Samsung Camera, on an iPhone it opens iPhone Camera, on a Pixel it opens Google Camera. Same action, different behaviour per device.

Worked Example — A Game Character System

► Step 1: Inheritance for shared behaviour

game_inheritance.py

```
class Character:
    def move(self):
        print("Moving...")

class Warrior(Character):
    pass

class Mage(Character):
    pass

class Archer(Character):
    pass
```

Every character can now move() with zero duplicated code — that's inheritance doing its job.

► Step 2: Polymorphism for unique behaviour

game_polymorphism.py

```
class Character:
    def attack(self):
        print("Basic attack")

class Warrior(Character):
    def attack(self):
        print("Sword Slash")

class Mage(Character):
    def attack(self):
        print("Fireball")

class Archer(Character):
    def attack(self):
        print("Arrow Shot")

characters = [Warrior(), Mage(), Archer()]

for c in characters:
    c.attack()
```

OUTPUT

```
Sword Slash
Fireball
Arrow Shot
```

The loop never checks isinstance(c, Warrior) or similar — it just calls c.attack() and Python resolves the correct version automatically.

INHERITANCE WITHOUT POLYMORPHISM?

Yes — possible. If a subclass never overrides a method, there's no polymorphic behaviour, just reused code.

POLYMORPHISM WITHOUT INHERITANCE?

Also yes, in Python — via duck typing, shown next.

► Duck typing: polymorphism without a shared parent class

duck_typing.py

```
class Dog:
    def sound(self):
        print("Woof!")

class Car:
    def sound(self):
        print("Horn!")

objects = [Dog(), Car()]

for obj in objects:
    obj.sound()
```

OUTPUT

Woof!
Horn!

Dog and Car are completely unrelated classes, but because both provide a `sound()` method, the exact same loop works on both. This is polymorphism achieved purely through a shared method name — no common ancestor required.

When to Reach for Each

USE INHERITANCE WHEN...

- Classes have an “is-a” relationship
- You want to reuse common code
- Multiple classes share the same attributes or methods

USE POLYMORPHISM WHEN...

- Different objects should respond to the same call differently
- You want to avoid long if/elif type-checking chains
- You want code that's easy to extend with new classes

6. Types of Polymorphism

OOP recognises two main categories of polymorphism. Python leans almost entirely on the second one.

Type	Also Called	Decision Made	In Python?
Compile-time Polymorphism	Static Polymorphism	Before the program runs	No true overloading
Run-time Polymorphism	Dynamic Polymorphism	While the program runs	Yes, via overriding
Operator Overloading	Special form of polymorphism	Depends on operand type	Yes

6.1 Compile-Time (Static) Polymorphism

The method to run is decided before the program executes, typically through method overloading. Languages like C++ and Java support this directly:

Calculator.cpp

```
class Calculator
{
public:
    int add(int a, int b)
    {
        return a + b;
    }

    int add(int a, int b, int c)
    {
        return a + b + c;
    }
};
```

The compiler already knows which add() to call based on argument count — c.add(2,3) calls the first, c.add(2,3,4) calls the second.

DOES PYTHON SUPPORT METHOD OVERLOADING?

No. Defining add() twice simply overwrites the first definition entirely:

calculator_no_overload.py

```
class Calculator:
    def add(self, a, b):
        return a + b

    def add(self, a, b, c):
        return a + b + c

c = Calculator()
c.add(2, 3)  # TypeError
```

OUTPUT

`TypeError` (the two-argument `add()` no longer exists)

Python programmers get overloading-like flexibility through variable-length arguments instead:

`calculator_varargs.py`

```
class Calculator:
    def add(self, *numbers):
        return sum(numbers)

c = Calculator()
print(c.add(2, 3))
print(c.add(2, 3, 4))
print(c.add(2, 3, 4, 5))
```

OUTPUT

```
5
9
14
```

This is not true method overloading — it's a Python idiom using `*args` to accept a variable number of arguments.

6.2 Run-Time (Dynamic) Polymorphism

This is the flavour Python uses constantly. The method that executes is decided while the program is running, through method overriding:

`runtime_poly.py`

```
class Animal:
    def sound(self):
        print("Some sound")

class Dog(Animal):
    def sound(self):
        print("Woof!")

class Cat(Animal):
    def sound(self):
        print("Meow!")

animals = [Dog(), Cat()]

for animal in animals:
    animal.sound()
```

OUTPUT

```
Woof!
Meow!
```

► How Python finds the correct method

When `animal.sound()` is called on a `Dog` instance, Python searches for `sound()` in this order:



Since Dog defines its own sound(), that version runs immediately. If it didn't, Python would fall back to Animal.sound(). This lookup order is called the Method Resolution Order (MRO).

6.3 Operator Overloading

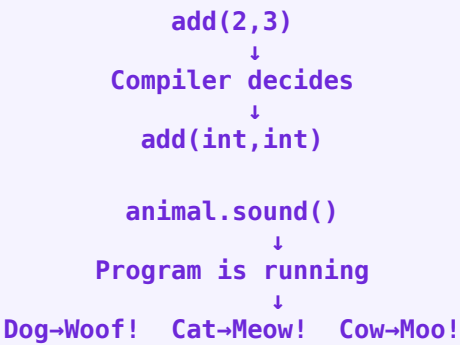
Python also expresses polymorphism through operators — the very same + behaves differently depending on the operand types:

operator_overload.py

```
print(5 + 3)           # 8
print("Hello " + "World") # Hello World
print([1, 2] + [3, 4]) # [1, 2, 3, 4]
```

Operand Types	Behaviour of +
Numbers	Addition
Strings	Concatenation
Lists	Joining (concatenation)

Compile-Time vs. Run-Time, Visualised



7. One-Sentence Takeaways

Inheritance answers: “What does this class inherit?”

Polymorphism answers: “What happens when I call the same method on different objects?”

THE FULL DEFINITION

Polymorphism allows one interface — such as a common method name — to have many implementations, enabling different objects to respond differently to the same method call, without the caller ever needing to know the object's specific type.